

Nombre

CC2008 – Introduccion a la programación orientada a objetos

Laboratorio 2 – Arreglos y excepciones



Agencia de Detectives – Caso misterioso

Requisitos funcionales

- Solicitar al iniciar el programa el nombre del caso, su código de identificación y el nombre del detective responsable.
- Administrar un máximo de cinco ubicaciones mediante un arreglo básico de objetos.
- Almacenar para cada ubicación su código, nombre, dirección o descripción del lugar, nivel de riesgo y estado.
- Validar que el nivel de riesgo esté entre 1 y 10. Si se proporciona un valor fuera de ese intervalo, generar una `IllegalArgumentException`.
- Permitir registrar una ubicación en una posición elegida por el usuario, verificando los límites del arreglo, la disponibilidad de la posición y la validez de los datos.
- Mantener con valor null las posiciones sin ubicaciones y comprobar esta condición antes de utilizar un objeto.
- Mostrar todas las ubicaciones registradas y las posiciones que ocupan, omitiendo los espacios vacíos.
- Consultar una ubicación mediante su posición. Informar cuando la posición sea inválida o esté vacía.
- Modificar el nivel de riesgo y el estado de una ubicación existente, validando los nuevos datos.
- Descartar una ubicación y dejar su posición nuevamente con valor null.
- Administrar las pistas mediante un `ArrayList` de objetos `Pista`, sin utilizar `List`.
- Almacenar para cada pista su código, descripción, tipo de evidencia, nivel de importancia y nivel de confiabilidad.
- Validar que la importancia esté entre 1 y 10 y la confiabilidad entre 0 y 100. Generar una `IllegalArgumentException` cuando alguno de estos valores sea inválido.
- Registrar pistas sin permitir códigos repetidos.

- Mostrar todas las pistas registradas e informar cuando la colección esté vacía.
- Buscar una pista por su código mediante un recorrido del ArrayList.
- Modificar la información de una pista existente, aplicando las mismas validaciones utilizadas al registrarla y evitando códigos duplicados.
- Eliminar una pista del ArrayList después de localizarla por su código.
- Generar un reporte con la cantidad de ubicaciones, los espacios disponibles, la ubicación de mayor riesgo, la cantidad de pistas, la pista de mayor importancia, la pista de mayor confiabilidad y el promedio de importancia.
- Realizar los cálculos solamente con los objetos registrados. Evitar acceder a elementos inexistentes o calcular promedios cuando no existan pistas.
- Capturar InputMismatchException cuando el usuario escriba un dato incompatible con una entrada numérica. Limpiar la entrada y permitir otro intento.
- Utilizar bloques try-catch para manejar los errores de entrada y las excepciones generadas durante las operaciones.
- Utilizar finally para realizar acciones necesarias tanto si una operación tiene éxito como si ocurre una excepción.
- Mantener el programa en funcionamiento después de manejar una entrada incorrecta o una excepción.
- Mostrar un menú con las 13 opciones solicitadas. La opción Nuevo caso debe reemplazar el caso actual por uno sin ubicaciones ni pistas; la opción Salir debe finalizar el programa.

Clases y atributos

Clase Ubicación

Atributo	Tipo	Proposito
Código	String	Identifica la ubicación
Nombre	String	Guarda el nombre del lugar
Dirección	String	Guarda la dirección del lugar
nivelRiesgo	Int	Representa la riesgo del lugar mediante un valor de 1 y 10
Estado	String	Indica la situación de la ubicación.

Ubicacion(String codigo, String nombre, String direccion, int nivelRiesgo, String estado): constructor público que recibe los datos iniciales. Valida los textos obligatorios y utiliza actualizar para comprobar el riesgo y el estado. Puede generar IllegalArgumentException.

getNivelRiesgo(): int: método público sin parámetros que devuelve el riesgo. Permite comparar ubicaciones durante la elaboración del reporte.

actualizar(int nivelRiesgo, String estado): void: método público que modifica el riesgo y el estado después de validar ambos valores. No devuelve información. Puede generar IllegalArgumentException y, si los datos son inválidos, conserva los valores anteriores.

toString(): String: método público sin parámetros que devuelve una representación de todos los datos de la ubicación para mostrarlos en pantalla.

Clase pista

Atributo	Tipo	Proposito
Código	String	Identifica la pista y permite buscarla sin duplicar códigos dentro del caso
descripcion	String	Explica que e encontró
tipoEvidencia	String	Clasifica la evidencia
nivelImportancia	Int	Expresa la relevancia de la pista mediante un valor de 1 y 10
nivelConfiabilidad	Int	Expresa su confiabilidad mediante un valor entre 0 y 100

Pista(String codigo, String descripcion, String tipoEvidencia, int nivelImportancia, int nivelConfiabilidad): constructor público que recibe y valida los cinco datos. Genera

IllegalArgumentException si hay textos vacíos, importancia fuera de 1 a 10 o confiabilidad fuera de 0 a 100.

getCodigo(): String: método público sin parámetros que devuelve el código para realizar búsquedas y comprobar duplicados.

getNivelImportancia(): int: método público sin parámetros que devuelve la importancia para obtener el máximo y calcular el promedio.

getNivelConfiabilidad(): int: método público sin parámetros que devuelve la confiabilidad para comparar las pistas.

toString(): String: método público sin parámetros que devuelve todos los datos de la pista en un texto.

Clase Caso

Atributo	Tipo	Propósito
Nombre	String	Guarda el nombre del caso
código	String	Guarda su código de identificación
Detective	String	Identifica el responsable de la identificación
Ubicaciones	Ubicación[]	Almacena un máximo de cinco ubicaciones, las posiciones disponibles contienen null
pistas	ArrayList<pista>	Almacena una cantidad variable de pistas y permite agregarlas, reemplazarlas y eliminarlas

Caso(String nombre, String codigo, String detective): constructor público que valida los datos del caso, crea el arreglo de cinco posiciones y crea el ArrayList vacío. Puede generar IllegalArgumentException.

validarPosicion(int posicion): void: método privado que comprueba si una posición está entre 0 y 4. Genera IndexOutOfBoundsException cuando queda fuera del arreglo.

registrarUbicacion(int posicion, Ubicacion ubicacion): void: método público que almacena una ubicación en la posición indicada. Comprueba los límites, la disponibilidad y que el objeto no sea null. Puede generar IndexOutOfBoundsException, IllegalStateException o IllegalArgumentException.

consultarUbicacion(int posicion): Ubicacion: método público que devuelve la ubicación de una posición. Genera IndexOutOfBoundsException si la posición es inválida o IllegalStateException si está vacía.

consultarUbicaciones(): String: método público sin parámetros que recorre el arreglo y devuelve las ubicaciones existentes con sus posiciones. Si no hay ninguna, devuelve un mensaje informativo.

modificarUbicacion(int posicion, int nivelRiesgo, String estado): void: método público que obtiene una ubicación y actualiza su riesgo y estado. Puede generar las excepciones de consultarUbicacion y actualizar.

descartarUbicacion(int posicion): void: método público que comprueba la existencia de una ubicación y asigna null a su posición. Puede generar IndexOutOfBoundsException o IllegalStateException.

buscarIndicePista(String codigo): int: método privado que recorre el ArrayList y devuelve el índice de la pista encontrada o -1 si no existe. Ignora diferencias entre mayúsculas y minúsculas y espacios al inicio o al final. Genera IllegalArgumentException si el código está vacío o es null.

registrarPista(Pista pista): void: método público que agrega una pista válida si su código no existe. Genera IllegalArgumentException si recibe null o un código repetido.

buscarPista(String codigo): Pista: método público que devuelve la pista correspondiente al código. Genera IllegalArgumentException si el código es inválido o no existe.

consultarPistas(): String: método público sin parámetros que devuelve todas las pistas o un mensaje si el ArrayList está vacío.

modificarPista(String codigoActual, Pista nuevaPista): void: método público que localiza la pista original, verifica que la nueva no sea null y comprueba que su código no corresponda a otra pista. Después reemplaza el objeto en el mismo índice. Puede generar IllegalArgumentException.

eliminarPista(String codigo): void: método público que localiza la pista y elimina su elemento del ArrayList. Genera IllegalArgumentException si el código es inválido o no se encuentra.

generarReporte(): String: método público sin parámetros que calcula y devuelve los siete resultados solicitados. Omite las posiciones null y evita obtener máximos o promedios de una colección vacía. En caso de empate, conserva el primer objeto encontrado.

toString(): String: método público sin parámetros que devuelve el nombre del caso, su código y el detective responsable.

Clase Principal

Atributo	Tipo	Proposito
Entrada	Scanner	Lee los datos que el usuario escribe en consola
caso	Caso	Mantiene la referencia al caso actual y permite reemplazarlo al seleccionar al nuevo caso

Principal(): constructor privado sin parámetros que crea el Scanner y asigna null al caso antes de solicitar sus datos.

main(String[] args): void: método público y estático que inicia el programa. Crea una instancia de Principal y llama a ejecutar. Recibe los argumentos de ejecución, aunque no los necesita.

ejecutar(): void: método privado sin parámetros que solicita el primer caso, mantiene activo el menú y ejecuta las opciones. Captura los errores de las operaciones y cierra el Scanner en finally al terminar.

mostrarMenu(): void: método privado sin parámetros que imprime los datos del caso actual y las 13 opciones disponibles.

crearCaso(): Caso: método privado sin parámetros que solicita el nombre, código y detective, y devuelve un Caso nuevo.

leerUbicacion(): Ubicacion: método privado sin parámetros que solicita los datos de una ubicación y devuelve el objeto construido. Los valores inválidos pueden provocar IllegalArgumentException, que se maneja en ejecutar.

leerPista(): Pista: método privado sin parámetros que solicita los datos de una pista y devuelve el objeto construido. Los valores inválidos pueden provocar IllegalArgumentException, que se maneja en ejecutar.

leerTexto(String mensaje): String: método privado que muestra una solicitud y devuelve un texto sin espacios al inicio o al final. Repite la solicitud cuando la entrada está vacía.

leerEntero(String mensaje): int: método privado que solicita y devuelve un número entero. Captura InputMismatchException, limpia la línea mediante finally y permite volver a intentarlo.

Diseno:

